Nifty100 Financial Intelligence Platform

Analyst Guide

Internship Project Documentation

Project Name: Nifty100 Financial Intelligence Platform

Author: Prashanth Reddy

Duration: 45-Day Agile Development Sprint

Version: 1.0

Date: 01 August 2026

Purpose of this Guide

This guide provides complete documentation for using the Nifty100 Financial Intelligence Platform. It is intended for analysts, developers, reviewers, and end users who want to understand, operate, and explore the project's dashboard, analytics engine, REST API, and generated reports.

The document explains how to install and run the application, navigate the Streamlit dashboard, generate company tearsheets, use the FastAPI endpoints, interpret financial metrics, execute the test suite, and troubleshoot common issues.

Project Overview

The Nifty100 Financial Intelligence Platform is an end-to-end financial analytics system built to analyze the financial performance of Nifty 100 companies using publicly available financial statements.

The project follows a complete data engineering and analytics workflow:

- Extract financial data from Excel datasets.
- Validate and clean the data using an ETL pipeline.
- Store processed data in a SQLite database.
- Compute 50+ financial KPIs.
- Perform company screening, peer comparison, sector analysis, clustering, and valuation.
- Generate Excel reports and PDF tearsheets.
- Provide an interactive Streamlit dashboard.
- Expose financial data through a FastAPI REST API with OpenAPI documentation.

The platform enables users to evaluate company fundamentals, compare businesses within sectors and peer groups, screen companies using financial metrics, and access historical financial information through both the dashboard and REST API.

Project Architecture

The Nifty100 Financial Intelligence Platform follows a modular architecture that separates data ingestion, analytics, reporting, dashboard visualization, and API services into independent components. This design improves maintainability, scalability, testing, and future enhancements.

The complete workflow begins with raw financial datasets and progresses through ETL, analytics, reporting, visualization, and API delivery.

Technology Stack

The platform is developed entirely using Python and open-source technologies.

Component	Technology Used
Programming Language	Python 3
Database	SQLite
Data Processing	Pandas, NumPy
Data Visualization	Plotly
Machine Learning	Scikit-learn (KMeans Clustering)
Dashboard	Streamlit
REST API	FastAPI
API Documentation	OpenAPI (Swagger UI & ReDoc)
Testing	Pytest
PDF Generation	ReportLab
Development Environment	Visual Studio Code

Project Modules

The platform consists of several independent modules that work together.

ETL Module

Responsible for loading, validating, normalizing, and storing financial datasets into the SQLite database.

Analytics Module

Computes profitability, leverage, growth, efficiency, cash flow, valuation, and quality metrics for every company.

Reporting Module

Generates Excel reports, portfolio summaries, sector reports, company tearsheets, and analytical outputs.

Dashboard Module

Provides an interactive Streamlit application for exploring company financials, peer comparisons, trends, sectors, and screeners.

REST API Module

Exposes financial data through 16 REST endpoints with automatically generated OpenAPI documentation, enabling external applications to access project data programmatically.

Testing & Quality Assurance Module

Ensures correctness of ETL, KPI calculations, API responses, and data validation rules through an automated Pytest test suite.

Installation & Running the Project

This chapter explains how to install, configure, and run the Nifty100 Financial Intelligence Platform. Ensure that all project dependencies are installed before executing any module.

System Requirements

The recommended development environment is:

- Operating System: Windows 10/11 or Linux
- Python 3.11 or later
- SQLite 3
- Git
- Visual Studio Code (recommended)

Installing the Project

Step 1 – Clone the Repository

```
git clone <repository-url>
cd Nifty100_Financial_Intelligence
```

Step 2 – Create a Virtual Environment

```
python -m venv .venv
```

Activate the virtual environment.

Windows

```
.venv\Scripts\activate
```

Linux / macOS

```
source .venv/bin/activate
```

Step 3 – Install Dependencies

```
pip install -r requirements.txt
```

This installs all required Python libraries, including:

- Pandas
- NumPy
- Plotly
- Streamlit
- FastAPI
- Uvicorn
- Scikit-learn
- ReportLab
- Pytest
- SQLite support libraries

Running the ETL Pipeline

The ETL module loads the Excel datasets, validates the data, and populates the SQLite database.

Run the ETL process using the appropriate loader script.

Example:

```
python src/etl/db_loader.py
```

After successful execution:

- SQLite database (`nifty100.db`) is created/updated.
- Financial datasets are loaded into database tables.
- Validation reports are generated.
- Load audit reports are generated.

Running the Streamlit Dashboard

Launch the dashboard using:

```
streamlit run src/dashboard/app.py
```

The dashboard starts on:

```
http://localhost:8501
```

Users can navigate through the available dashboard pages to analyze companies, financial metrics, sectors, peer groups, and reports.

Running the FastAPI Server

Start the REST API using Uvicorn.

```
uvicorn src.api.main:app --reload --port 8000
```

The API becomes available at:

```
http://127.0.0.1:8000
```

Interactive API documentation:

Swagger UI

```
http://127.0.0.1:8000/docs
```

ReDoc

```
http://127.0.0.1:8000/redoc
```

Running the Test Suite

Execute all automated tests using:

```
pytest tests/
```

To generate the HTML report:

```
pytest tests/ --html=reports/pytest_report.html --self-contained-html
```

The generated report summarizes the complete test execution, including passed tests, failed tests, execution time, and overall project

quality status.

Streamlit Dashboard Overview

The Nifty100 Financial Intelligence Platform provides an interactive **Streamlit dashboard** that enables users to explore financial information, compare companies, analyze industry sectors, and screen investment opportunities.

The dashboard is designed for analysts, investors, students, and researchers who require an intuitive interface for financial analysis without writing SQL queries or Python code.

After launching the application, users can navigate between dashboard pages using the left sidebar.

Dashboard Pages

The application consists of **8 interactive pages**, each focusing on a specific aspect of financial analysis.

Page	Purpose
Home	Overall summary of the Nifty100 universe
Company Profile	Detailed financial analysis of an individual company
Financial Screener	Filter companies using financial metrics
Peer Comparison	Compare companies within the same peer group
Trend Analysis	Analyze multi-year financial trends
Sector Analysis	Compare companies across sectors
Capital Allocation	Visualize capital allocation patterns
Annual Reports	Access company annual report documents

Home Dashboard

The Home page provides a high-level overview of the project and serves as the landing page for the dashboard.

Features

- Introduction to the platform
- Navigation guidance
- Entry point to all dashboard pages
- Clean and responsive interface

Company Profile

The Company Profile page provides a comprehensive financial overview of an individual company.

Users can search and select a company to view its financial information.

Information Displayed

- Company name
- Company logo
- Sector and sub-sector
- Market capitalization category
- Financial ratios
- Historical financial statements
- Business analysis
- Pros and Cons
- Annual report references

This page helps users evaluate the financial

Advanced Dashboard Features

This chapter describes the remaining dashboard pages and explains how they assist users in performing financial analysis across companies, peer groups, sectors, and historical data.

Peer Comparison

The Peer Comparison page enables users to compare a company with other businesses belonging to the same peer group.

Instead of analyzing a company in isolation, this page highlights how it performs relative to its competitors using financial KPIs and visual comparisons.

Features

- Peer group selection
- Company comparison
- Radar chart visualization
- Financial KPI comparison
- Percentile ranking
- Benchmark company comparison

This page helps analysts understand whether a company is outperforming or underperforming its industry peers.

Trend Analysis

The Trend Analysis page visualizes historical financial performance across multiple years.

Users can analyze long-term trends rather than relying on a single financial year.

Available Trend Metrics

- Revenue
- Net Profit
- Operating Profit
- Return on Equity (ROE)
- Return on Capital Employed (ROCE)
- Free Cash Flow
- Debt-to-Equity Ratio
- Earnings Per Share (EPS)

Features

- Multi-year trend charts
- Interactive Plotly visualizations
- Year selection
- Company comparison
- Historical KPI analysis

Trend analysis helps identify consistent growth, declining performance, and long-term business stability.

Sector Analysis

The Sector Analysis page compares companies belonging to the same industry sector.

Instead of focusing on one company, users can evaluate the overall financial characteristics of an entire sector.

Features

- Sector selection
- Sector-wise company listing
- Median financial KPIs
- Market capitalization comparison

- Company count by sector
- Sector performance comparison

This page helps users identify strong and weak sectors within the Nifty100 universe.

Capital Allocation

The Capital Allocation page classifies companies based on how they generate and utilize cash.

The platform analyzes operating, investing, and financing cash flows to identify capital allocation patterns.

Capital Allocation Categories

- Reinvestor
- Shareholder Returns
- Cash Accumulator
- Growth Funded by Debt
- Distress Signal
- Liquidating Assets
- Mixed
- Pre-Revenue

Features

- Treemap visualization
- Company classification
- Cash flow pattern analysis
- Interactive company selection

These classifications help analysts understand management's capital allocation strategy.

Annual Reports

The Annual Reports page provides direct access to company annual report documents.

Users can select a company and quickly access available annual reports without manually searching external websites.

Features

- Company selection
- Available report years
- Direct annual report links
- Document availability

This page simplifies financial research by centralizing annual report references within the dashboard.

Dashboard Best Practices

For an effective analysis workflow, users should follow the recommended sequence below:

1. Start with the **Home** page for an overview.
2. Use the **Financial Screener** to shortlist companies.
3. Analyze shortlisted companies using the **Company Profile** page.
4. Compare them with competitors in **Peer Comparison**.
5. Review long-term financial performance in **Trend Analysis**.
6. Evaluate industry performance using **Sector Analysis**.
7. Understand management decisions through **Capital Allocation**.
8. Refer to **Annual Reports** for detailed company disclosures.

Following this workflow enables users to move from high-level screening to detailed company analysis in a structured and efficient manner.

Report Generation & Project Outputs

The Nifty100 Financial Intelligence Platform automatically generates analytical reports that help users interpret financial data without manually performing calculations.

These reports are generated from the analytics engine and are stored within the project directories for future reference and sharing.

Company Tearsheets

A Company Tearsheet is a concise PDF report that summarizes the financial health and key performance indicators of a single company.

Each tearsheet is generated automatically for companies with sufficient historical financial data.

Information Included

- Company name
- Sector and Sub-sector
- Latest financial year
- Key financial KPIs
- Revenue and Profit trends
- ROE and ROCE
- Debt-to-Equity Ratio
- Free Cash Flow
- Radar chart
- Financial highlights

Generated files are stored in:

```
reports/tearsheets/
```

These reports provide a quick overview of a company's financial performance and are useful for presentations, investment research, and management reviews.

Sector Reports

The platform also generates sector-wise PDF reports summarizing companies belonging to the same industry.

Each report includes:

- Sector overview
- Company count
- Median financial KPIs
- Sector performance summary
- Company comparison table
- Financial highlights for all companies within the sector

Generated files are stored in:

```
reports/sector/
```

These reports help analysts compare businesses within a specific industry.

Portfolio Summary Report

The Portfolio Summary Report provides a consolidated overview of all companies included in the project.

The report presents one page per company in alphabetical order.

Each page includes:

- Company information
- Sector
- Top financial KPIs

- Financial trend indicators
- Performance summary

Generated file:

```
reports/portfolio/portfolio_summary.pdf
```

This report is useful for obtaining a high-level overview of the complete Nifty100 dataset.

Generated Analytical Outputs

During execution, the platform automatically generates several reports and datasets.

Excel Reports

- Peer Comparison Workbook
- Portfolio Statistics
- Cash Flow Intelligence
- Screener Results

CSV Reports

- Cluster Labels
- Portfolio Statistics
- Performance Notes
- Validation Failures
- Load Audit
- Screener Outputs
- Capital Allocation Results

Charts

The reporting engine automatically generates visualizations, including:

- Radar Charts
- Correlation Heatmaps
- Elbow Plot for KMeans Clustering

These charts support comparative analysis and model evaluation.

Exporting Reports

Generated reports can be accessed directly from their respective folders after execution.

Examples:

```
reports/tearsheets/
reports/sector/
reports/portfolio/
reports/radar_charts/
output/
```

These files can be shared with analysts, faculty, project reviewers, or stakeholders without requiring the Streamlit dashboard or API.

Benefits of Automated Reporting

The automated reporting system offers several advantages:

- Eliminates repetitive manual analysis.
- Produces standardized reports for every company.
- Improves consistency and accuracy.
- Supports investment research and presentations.
- Simplifies financial decision-making.

- Enables quick access to company and sector insights.

By combining dashboard visualizations with downloadable reports, the platform provides both interactive exploration and professional offline documentation.

REST API User Guide

The Nifty100 Financial Intelligence Platform provides a REST API developed using **FastAPI**. The API enables external applications, developers, and analysts to access financial data programmatically without using the Streamlit dashboard.

The API follows REST principles and returns responses in JSON format.

Starting the API Server

Run the following command from the project root directory:

```
uvicorn src.api.main:app --reload --port 8000
```

After successful startup, the API will be available at:

```
http://127.0.0.1:8000
```

Interactive API Documentation

FastAPI automatically generates interactive API documentation.

Swagger UI

```
http://127.0.0.1:8000/docs
```

Swagger UI allows users to:

- Browse all available endpoints
- Execute API requests directly from the browser
- View request parameters
- View JSON responses

ReDoc

```
http://127.0.0.1:8000/redoc
```

ReDoc provides a clean, documentation-style view of all API endpoints and their descriptions.

Major API Endpoints

The platform exposes endpoints covering company information, financial statements, screeners, sectors, peer groups, valuation, portfolio statistics, and supporting documents.

Health Check

Returns application status and database information.

```
curl http://127.0.0.1:8000/api/v1/health
```

List Companies

Returns all available companies with their latest financial information.

```
curl http://127.0.0.1:8000/api/v1/companies
```

Company Profile

Returns complete financial information for a single company.

Example:

```
curl http://127.0.0.1:8000/api/v1/companies/TCS
```

Profit & Loss History

Returns historical Profit & Loss data for a company.

```
curl "http://127.0.0.1:8000/api/v1/companies/TCS/pl"
```

Balance Sheet

Returns Balance Sheet history.

```
curl "http://127.0.0.1:8000/api/v1/companies/TCS/bs"
```

Cash Flow

Returns Cash Flow history.

```
curl "http://127.0.0.1:8000/api/v1/companies/TCS/cashflow"
```

Financial Ratios

Returns calculated KPIs for all available years.

```
curl "http://127.0.0.1:8000/api/v1/companies/TCS/ratios"
```

Financial Screener

Filters companies based on financial criteria.

Example:

```
curl "http://127.0.0.1:8000/api/v1/screener?min_roe=15&max_de=1"
```

Sector Analysis

Returns all available sectors.

```
curl http://127.0.0.1:8000/api/v1/sectors
```

Returns companies belonging to a specific sector.

```
curl "http://127.0.0.1:8000/api/v1/sectors/Information%20Technology/companies"
```

Peer Comparison

Returns companies belonging to a peer group.

```
curl "http://127.0.0.1:8000/api/v1/peers/Private%20Banks"
```

Portfolio Statistics

Returns percentile statistics across portfolio KPIs.

```
curl http://127.0.0.1:8000/api/v1/portfolio/stats
```

Company Documents

Returns annual report links for a company.

```
curl http://127.0.0.1:8000/api/v1/companies/TCS/documents
```

API Response Format

All successful requests return data in JSON format.

Example:

```
{
    "company_name": "Tata Consultancy Services",
    "broad_sector": "Information Technology",
    "return_on_equity_pct": 48.21,
    "return_on_capital_employed_pct": 61.73
}
```

Errors are returned using standard HTTP status codes.

Examples:

- 200 – Success
- 404 – Resource not found
- 422 – Invalid request parameters

API Best Practices

- Verify the API server is running before sending requests.
- Use valid company tickers when requesting company-specific data.
- Refer to Swagger UI for parameter descriptions and example values.
- Use query parameters to reduce unnecessary data transfer.
- Prefer the REST API for programmatic access and automation.

The FastAPI service provides a lightweight and efficient interface for accessing financial data, making it suitable for integration with dashboards, analytics tools, and external applications.

Testing & Troubleshooting Guide

This chapter explains how to validate the project using automated tests and provides solutions to common issues that may occur while running the ETL pipeline, dashboard, or REST API.

Running the Test Suite

The project includes automated tests for ETL, KPI calculations, and REST API endpoints using **Pytest**.

Run all tests using:

```
pytest tests/
```

To execute tests with detailed output:

```
pytest tests/ -v
```

The project currently includes tests covering:

- ETL Loader

- Data Normalization
- Data Validation
- Financial Ratio Calculations
- CAGR Calculations
- Cash Flow Analytics
- Health API
- Companies API
- Screener API
- Sector API

These tests help ensure that the application's core functionality remains reliable after future code changes.

Generating the HTML Test Report

Generate an HTML summary of all executed tests using:

```
pytest tests/ --html=reports/pytest_report.html --self-contained-html
```

The generated report includes:

- Total tests executed
- Passed tests
- Failed tests
- Skipped tests
- Execution time
- Detailed results for each test case

The report is saved as:

```
reports/pytest_report.html
```

Common Issues & Solutions

1. ModuleNotFoundError

Problem

```
ModuleNotFoundError: No module named 'src'
```

Solution

Run commands from the project root directory or configure the Python path correctly.

---

2. Database Not Found

Problem

```
sqlite3.OperationalError:
no such table
```

Solution

- Ensure `nifty100.db` exists.
- Execute the ETL pipeline before running the dashboard or API.
- Verify the database path in the project configuration.

---

3. Streamlit Does Not Start

Problem

```
streamlit : command not found
```

Solution

Install Streamlit using:

```
pip install streamlit
```

Verify installation:

```
streamlit --version
```

---

4. FastAPI Server Does Not Start

Problem

```
ModuleNotFoundError
```

or

```
Address already in use
```

Solution

- Verify all dependencies are installed.
- Ensure port **8000** is not occupied.
- Stop any existing Uvicorn process before restarting the server.

Start the server using:

```
uvicorn src.api.main:app --reload --port 8000
```

---

5. API Returns HTTP 404

Possible Causes

- Incorrect company ticker
- Invalid API endpoint
- Resource does not exist

Solution

- Verify the endpoint URL.
- Use valid company tickers.
- Refer to Swagger UI for endpoint documentation.

---

6. Dashboard Displays No Data

Possible Causes

- SQLite database not populated
- Missing CSV output files
- ETL process not executed

Solution

- Run the ETL pipeline.
- Verify the database contains data.
- Confirm all required output files have been generated.

---

7. PDF Reports Not Generated

Possible Causes

- Missing ReportLab package
- Invalid output directory
- Missing company financial data

Solution

- Install ReportLab.
- Verify output folders exist.
- Ensure the selected company has sufficient financial data.

Best Practices

To ensure smooth operation of the platform:

- Always execute the ETL pipeline before using the dashboard or API.
- Keep project dependencies updated.
- Generate reports only after validating the database.
- Run the automated test suite after making code changes.
- Review the HTML test report before deployment.
- Maintain backups of the SQLite database before performing major updates.

Summary

The combination of automated testing, HTML test reporting, and troubleshooting procedures helps ensure that the Nifty100 Financial Intelligence Platform remains reliable, maintainable, and easy to operate.

Following the recommendations in this guide enables users to quickly identify issues, validate project functionality, and maintain a stable development environment.

Project Maintenance & Best Practices

Proper maintenance ensures that the Nifty100 Financial Intelligence Platform remains accurate, reliable, and easy to extend as new financial data becomes available.

This chapter provides recommendations for maintaining the project and describes the workflow for updating data, validating outputs, and extending functionality.

Updating Financial Data

Financial statements and supporting datasets are updated periodically. To refresh the project with the latest information:

Step 1

Replace or update the Excel files inside the data directory.

```
data/raw/
data/supporting/
```

Step 2

Run the ETL pipeline.

```
python src/etl/db_loader.py
```

This process:

- Loads the updated datasets.
- Normalizes company and year information.
- Validates the schema.
- Performs data quality checks.
- Updates the SQLite database.

Step 3

Recompute financial analytics.

Execute the analytics modules to regenerate:

- Financial Ratios
- CAGR Metrics
- Cash Flow KPIs
- Composite Quality Scores
- Capital Allocation
- Valuation Metrics
- Cluster Labels

Step 4

Regenerate reports.

Generate the latest analytical outputs, including:

- Company Tearsheets
- Sector Reports
- Portfolio Summary
- Radar Charts
- Excel Reports

These reports ensure that users always access the latest financial insights.

Running Validation

After updating the data, validate the project before deployment.

Recommended validation workflow:

```
pytest tests/
```

Generate the HTML report:

```
pytest tests/ --html=reports/pytest_report.html --self-contained-html
```

Review the report and ensure that all automated tests pass successfully before publishing updated results.

Deployment Checklist

Before deploying or sharing the project, verify the following:

- ETL pipeline executed successfully.
- SQLite database updated.
- Financial ratios regenerated.
- Reports generated successfully.
- Streamlit dashboard functioning correctly.
- FastAPI server running successfully.
- All automated tests passed.
- HTML test report generated.
- Documentation updated.

Future Enhancements

The project has been designed with a modular architecture, allowing future improvements without significant structural changes.

Potential enhancements include:

- Integration with live stock market APIs.
- Automatic financial statement updates.
- User authentication and role-based access.

- Portfolio tracking and watchlists.
- Advanced valuation models.
- AI-assisted financial analysis.
- Predictive analytics using machine learning.
- Cloud database integration.
- Docker containerization.
- CI/CD pipeline using GitHub Actions.

Conclusion

The Nifty100 Financial Intelligence Platform provides a complete financial analytics solution that combines ETL, data validation, financial ratio computation, screening, peer analysis, reporting, visualization, and REST API services within a single integrated application.

The platform enables analysts to transform raw financial statements into meaningful business insights through automated analytics, interactive dashboards, downloadable reports, and programmatic API access.

The modular architecture, automated testing, comprehensive documentation, and reporting capabilities make the project reliable, maintainable, and suitable for academic, research, and professional financial analysis.

This Analyst Guide serves as a reference for installing, operating, maintaining, and extending the platform, ensuring that users can efficiently utilize all features provided by the system.

Appendix

This appendix provides a quick reference to the project structure, commonly used commands, generated outputs, and useful resources for operating the Nifty100 Financial Intelligence Platform.

Project Directory Structure

```
NIFTY100_FINANCIAL_INTELLIGENCE/
|
├── config/
├── data/
├── db/
├── docs/
├── output/
├── performance/
├── reports/
├── src/
|   ├── analytics/
|   ├── api/
|   ├── dashboard/
|   ├── etl/
|   ├── reporting/
|   └── screener/
├── tests/
|   ├── api/
|   ├── etl/
|   └── kpi/
|
├── requirements.txt
├── README.md
└── nifty100.db
```

Frequently Used Commands

Install Dependencies

```
pip install -r requirements.txt
```

Run ETL

```
python src/etl/db_loader.py
```

Run Streamlit Dashboard

```
streamlit run src/dashboard/app.py
```

Dashboard URL:

```
http://localhost:8501
```

Run FastAPI Server

```
uvicorn src.api.main:app --reload --port 8000
```

API Base URL:

```
http://127.0.0.1:8000
```

Swagger UI:

```
http://127.0.0.1:8000/docs
```

ReDoc:

```
http://127.0.0.1:8000/redoc
```

Run Automated Tests

Execute the complete test suite:

```
pytest tests/
```

Verbose output:

```
pytest tests/ -v
```

Generate HTML Report:

```
pytest tests/ --html=reports/pytest_report.html --self-contained-html
```

Important Output Directories

Directory	Description
`reports/tearsheets/`	Company PDF Tearsheets
`reports/sector/`	Sector Summary Reports
`reports/portfolio/`	Portfolio Summary Report
`reports/radar_charts/`	Peer Comparison Radar Charts
`output/`	CSV, Excel, validation reports, and generated outputs
`docs/`	Project documentation and user guides
`tests/`	Automated unit and API tests

Generated Reports

The platform produces several analytical outputs, including:

- Company PDF Tearsheets
- Sector Reports
- Portfolio Summary Report
- Radar Charts
- Excel Reports
- Validation Reports
- Portfolio Statistics
- Cluster Labels
- HTML Test Report
- Performance Notes

References

The project utilizes publicly available financial information and open-source software.

Primary Data Sources

- NSE India
- BSE India
- Company Annual Reports

Technologies

- Python
- SQLite
- Pandas
- NumPy
- Plotly
- Streamlit
- FastAPI
- Scikit-learn
- ReportLab
- Pytest

Document Information

Project: Nifty100 Financial Intelligence Platform

Document: Analyst Guide

Version: 1.0

Author: Prashanth Reddy

Prepared For: Bluestock Fintech Internship Project

End of Document